

UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación a Distancia

Trabajo Práctico Integrador

**Gestión de Datos de Países en Python:
filtros, ordenamientos y estadísticas**

Materia: Programación 1

Alumno: **de Prado, Nicolás Germán**

DNI: **40.976.550**

Fecha de entrega: Junio 2026

Índice

1. Introducción	3
2. Marco Teórico	4
2.1 Listas	4
2.2 Diccionarios	4
2.3 Funciones	5
2.4 Condicionales	5-6
2.5 Ordenamientos	6
2.6 Estadísticas básicas	6-7
2.7 Archivos CSV	7-8
3. Flujo de Operaciones — Diagrama	9
4. Decisiones Técnicas	10
5. Dificultades y Conclusiones	11
6. Bibliografía / Webgrafía	12

1. Introducción

El presente informe acompaña el Trabajo Práctico Integrador de la materia Programación 1. El proyecto consiste en una aplicación de consola desarrollada en Python 3 que permite gestionar un dataset de países del mundo almacenado en el archivo `países.csv`.

El sistema está compuesto por dos archivos Python: `main.py`, que contiene el punto de entrada y el menú principal, y `funciones.py`, que agrupa todas las funciones del sistema. El usuario interactúa a través de un menú numérico con 12 opciones disponibles.

El objetivo del informe es explicar los conceptos teóricos aplicados en el desarrollo, describir las decisiones técnicas adoptadas y presentar el flujo de operaciones del sistema mediante un diagrama.

2. Marco Teórico

2.1 Listas

Una lista en Python es una estructura de datos ordenada y mutable que permite almacenar múltiples elementos en una sola variable. Se define con corchetes `[]` y puede crecer o achicarse dinámicamente. Los elementos se acceden por su índice (comenzando en 0) y se pueden recorrer con un bucle `for`.

Aplicación en el proyecto

En `funciones.py`, la lista `países` es la estructura central del sistema. Se inicializa vacía en `cargar_paises()` y se va llenando con cada fila del CSV. Todas las funciones del sistema reciben esta lista como parámetro y trabajan sobre ella.

Operación	Código en funciones.py
Crear lista vacía	<code>países = []</code>
Agregar elemento	<code>países.append(nuevo_pais)</code>
Recorrer lista	<code>for pais in países:</code>
Contar elementos	<code>len(países)</code>

2.2 Diccionarios

Un diccionario en Python es una colección de pares clave-valor, mutable y sin orden de índice numérico. Los valores se acceden por su clave, lo que hace el código más legible y menos propenso a errores que usar posiciones numéricas.

Aplicación en el proyecto

Cada país se representa como un diccionario con cuatro claves fijas. Este diccionario se construye dentro de `cargar_paises()` al leer cada fila del CSV, y también en `agregar_pais()` cuando el usuario ingresa un nuevo país manualmente.

Clave	Tipo	Descripción
<code>nombre</code>	<code>str</code>	Nombre del país
<code>poblacion</code>	<code>int</code>	Cantidad de habitantes
<code>superficie</code>	<code>int</code>	Superficie en km ²
<code>continente</code>	<code>str</code>	Continente al que pertenece

En `cargar_paises()`, los valores de `poblacion` y `superficie` se convierten explícitamente a entero con `int()` al momento de construir el diccionario, ya que `csv.DictReader` los devuelve como cadenas de texto por defecto.

2.3 Funciones

Una función es un bloque de código reutilizable que realiza una tarea específica. En Python se definen con `def`. Pueden recibir parámetros y devolver valores con `return`. El principio aplicado en este proyecto es 'una función = una responsabilidad'.

Funciones definidas en `funciones.py`

Función	Responsabilidad
<code>cargar_paises(archivo)</code>	Lee el CSV y devuelve la lista de diccionarios
<code>guardar_paises(archivo, paises)</code>	Escribe la lista en el CSV (opciones 11 y 12)
<code>agregar_pais(paises)</code>	Solicita datos al usuario y agrega un nuevo país a la lista
<code>actualizar_pais(paises)</code>	Busca un país por nombre exacto y actualiza población y superficie
<code>buscar_pais(paises)</code>	Búsqueda parcial por nombre, muestra todos los coincidentes
<code>filtrar_continente(paises)</code>	Muestra países del continente indicado
<code>filtrar_poblacion(paises)</code>	Muestra países dentro de un rango de población
<code>filtrar_superficie(paises)</code>	Muestra países dentro de un rango de superficie
<code>ordenar_nombre(paises)</code>	Ordena por nombre de forma ascendente
<code>ordenar_poblacion(paises)</code>	Ordena por población (ascendente o descendente)
<code>ordenar_superficie(paises)</code>	Ordena por superficie (ascendente o descendente)
<code>mostrar_estadisticas(paises)</code>	Calcula y muestra máximos, promedios y conteo por continente
<code>mostrar_menu()</code>	Imprime las 12 opciones del menú en consola

En `main.py`, la función `main()` integra todas las anteriores: carga el CSV al inicio, ejecuta el bucle del menú y llama a cada función según la opción seleccionada por el usuario.

2.4 Condicionales

Las estructuras condicionales (`if`, `elif`, `else`) permiten ejecutar distintos bloques de código según si una condición es verdadera o falsa. Son la base del control de flujo del programa.

Aplicación en el proyecto

Los condicionales aparecen en dos niveles del código:

- En `main.py`: el bloque `if/elif/else` dirige el flujo según la opción numérica ingresada por el usuario (opciones 1 a 12). Si la opción no corresponde a ninguna, muestra 'Opción inválida'.
- En `funciones.py`: se usan para validar que los campos no estén vacíos en `agregar_pais()`, verificar que el país exista en `actualizar_pais()`, controlar que haya resultados en las búsquedas y filtros, y validar la dirección de orden (A/D) en `ordenar_poblacion()` y `ordenar_superficie()`.

Ejemplo de condicional en `agregar_pais()`:

```
if nombre == '' or continente == '':  
    raise ValueError('No se permiten campos vacíos.')
```

Ejemplo de condicional en `actualizar_pais()`:

```
if pais['nombre'].lower() == nombre:  
    pais['poblacion'] = int(input('Nueva población: '))  
    pais['superficie'] = int(input('Nueva superficie: '))
```

2.5 Ordenamientos

Python incluye la función `sorted()` que recibe un iterable y devuelve una nueva lista ordenada sin modificar la original. Acepta el parámetro `key` (función que extrae el valor de comparación) y `reverse` (True para descendente, False para ascendente).

En el proyecto se usa una función `lambda` como `key`, que extrae el campo del diccionario sobre el que se quiere ordenar.

Los tres ordenamientos implementados en `funciones.py`

Función	Criterio	Dirección
<code>ordenar_nombre()</code>	<code>pais['nombre']</code>	Solo ascendente
<code>ordenar_poblacion()</code>	<code>pais['poblacion']</code>	A (ascendente) o D (descendente)
<code>ordenar_superficie()</code>	<code>pais['superficie']</code>	A (ascendente) o D (descendente)

Ejemplo de `ordenar_superficie()` con dirección elegida por el usuario:

```
opcion = input('A-Ascendente / D-Descendente: ').upper()  
if opcion == 'A':  
    ordenados = sorted(paises, key=lambda pais: pais['superficie'])  
elif opcion == 'D':  
    ordenados = sorted(paises, key=lambda pais: pais['superficie'],  
                        reverse=True)
```

2.6 Estadísticas básicas

La función `mostrar_estadisticas()` en `funciones.py` las combina para calcular todos los indicadores del dataset de una sola vez.

Función	Uso	Código en el proyecto
<code>max()</code>	País con mayor población	<code>max(paises, key=lambda pais: pais['poblacion'])</code>
<code>min()</code>	País con menor población	<code>min(paises, key=lambda pais: pais['poblacion'])</code>
<code>sum()</code>	Suma para calcular promedio	<code>sum(pais['poblacion'] for pais in paises)</code>
<code>len()</code>	Total de países	<code>len(paises)</code>
<code>round()</code>	Redondear promedios	<code>round(promedio_poblacion, 2)</code>

La cantidad de países por continente se calcula con un diccionario de conteo: se recorre la lista con un `for` y se incrementa el valor de la clave correspondiente al continente. Si el continente no existe aún en el diccionario, se inicializa en 1.

```
continentes = {}
for pais in paises:
    continente = pais['continente']
    if continente in continenten:
        continenten[continente] += 1
    else:
        continenten[continente] = 1
```

2.7 Archivos CSV

CSV (Comma-Separated Values) es un formato de texto plano donde cada línea representa un registro y los campos se separan por comas. Python incluye el módulo `csv` en su biblioteca estándar para leer y escribir este formato de forma estructurada.

Clases utilizadas en `csv_handler (funciones.py)`

- `csv.DictReader`: lee el archivo y devuelve cada fila como un diccionario, usando la primera línea (encabezado) como claves. Permite acceder a los campos por nombre (`fila['nombre']`) en lugar de por posición numérica.
- `csv.DictWriter`: escribe la lista de diccionarios en el archivo, generando automáticamente la cabecera y las filas. Se le indica el orden de los campos con `fieldnames`.

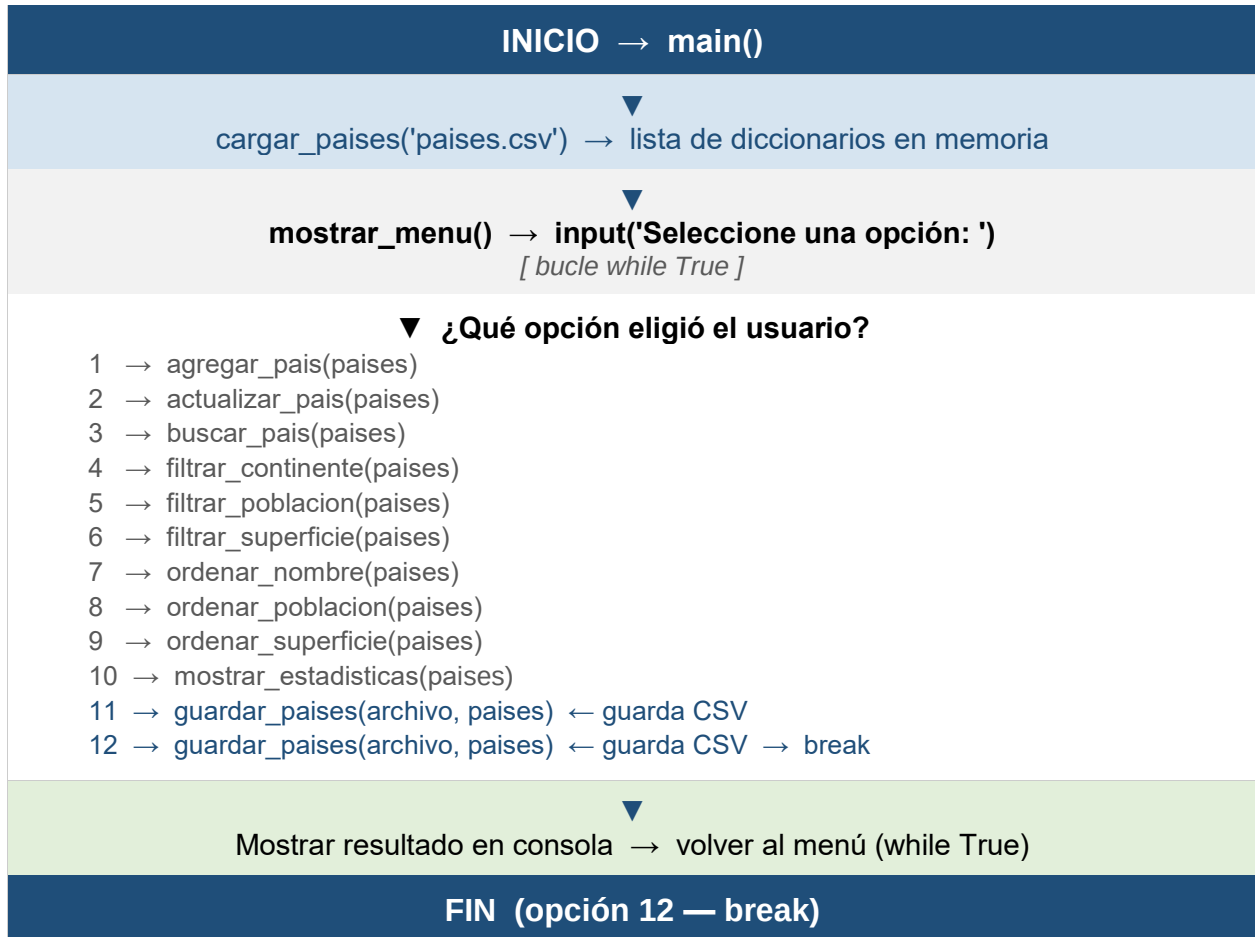
El archivo `paises.csv` tiene el siguiente formato con 10 países cargados:

```
nombre,poblacion,superficie,continente
Argentina,47000000,2780400,America
Brasil,216000000,8515767,America
China,1410000000,9596961,Asia
... (10 filas en total)
```

La función `cargar_paises()` envuelve la lectura en un bloque `try/except` que captura `FileNotFoundError` si el archivo no existe, y `Exception` para cualquier otro error inesperado, mostrando un mensaje claro al usuario en cada caso.

3. Flujo de Operaciones — Diagrama

El siguiente diagrama representa el flujo completo de ejecución del sistema. El programa inicia en `main()`, carga el CSV y entra en un bucle `while True` que solo termina cuando el usuario elige la opción 12. Las opciones 11 y 12 son las únicas que guardan el CSV.



Las opciones 11 y 12 son las únicas que llaman a `guardar_paises()` y persisten los cambios en el archivo CSV. La opción 12 además ejecuta `break` para salir del bucle `while`.

4. Decisiones Técnicas

4.1 Dos archivos: main.py y funciones.py

El código fue dividido en dos archivos con responsabilidades claras. main.py contiene únicamente el punto de entrada (función main()) y el bucle del menú. Todo el resto de la lógica se almacena en funciones.py, importado con `from funciones import *`.

Esta separación hace que main.py sea muy corto y fácil de leer: solo muestra el menú, lee la opción y delega en la función correspondiente.

4.2 Guardar solo en las opciones 11 y 12

A diferencia de guardar automáticamente tras cada operación, el sistema guarda el CSV únicamente cuando el usuario elige explícitamente 'Guardar cambios' (opción 11) o 'Salir' (opción 12). Esto permite al usuario explorar y modificar datos sin comprometerse hasta que decide guardar.

4.3 Búsqueda parcial por nombre

La función buscar_pais() usa el operador in para verificar si el texto ingresado está contenido en el nombre del país, ambos convertidos a minúsculas con lower(). Esto permite encontrar resultados escribiendo solo parte del nombre.

4.4 Manejo de errores con try/except

Las funciones que reciben entrada numérica del usuario (agregar_pais, actualizar_pais, filtrar_poblacion, filtrar_superficie) usan try/except ValueError para capturar el error si el usuario ingresa texto en lugar de un número, mostrando un mensaje claro sin detener el programa.

4.5 Dataset base: 10 países en países.csv

Nombre	Población	Superficie km²	Continente
Argentina	47.000.000	2.780.400	America
Brasil	216.000.000	8.515.767	America
Chile	19.600.000	756.102	America
Estados Unidos	340.000.000	9.833.517	America
España	49.000.000	505.990	Europa
Alemania	84.000.000	357.022	Europa
Francia	68.000.000	551.695	Europa
China	1.410.000.000	9.596.961	Asia
Japon	124.000.000	377.975	Asia
Australia	27.000.000	7.692.024	Oceania

5. Dificultades y Conclusiones

5.1 Dificultades encontradas

- Conversión de tipos al cargar el CSV: `csv.DictReader` devuelve todos los valores como cadenas de texto. Fue necesario convertir explícitamente `poblacion` y `superficie` a `int()` al construir cada diccionario en `cargar_paises()`.
- Comparación de texto: las búsquedas y el filtro por continente fallaban si el usuario escribía en distinto caso. Se resolvió convirtiendo ambas cadenas a minúsculas con `lower()` antes de comparar.
- Validación de campos vacíos: en `agregar_pais()`, si el usuario no ingresaba nombre o continente, el país quedaba con datos incompletos. Se resolvió con una condición que lanza `ValueError` antes de continuar.
- Decisión de cuándo guardar: al inicio se guardaba tras cada operación, lo que causaba escrituras innecesarias. Se optó por guardar solo en las opciones 11 y 12, dando control al usuario sobre cuándo persistir los cambios.

5.2 Conclusiones

El proyecto permitió integrar en forma práctica los conceptos fundamentales de Programación 1: listas, diccionarios, funciones, condicionales, ordenamientos, estadísticas y archivos CSV. Cada uno de estos conceptos tiene un rol concreto y visible en el sistema.

La separación entre `main.py` y `funciones.py` resultó muy útil para mantener el código organizado. El archivo `main.py` quedó limpio y legible, mientras que `funciones.py` concentra toda la lógica del programa en funciones con una única responsabilidad cada una.

Como reflexión final, este trabajo demuestra que con estructuras básicas de Python es posible construir aplicaciones funcionales y bien organizadas. La claridad del código y el buen manejo de errores son tan importantes como la funcionalidad en sí misma.

6. Bibliografía / Webgrafía

Documentación oficial de Python 3

- Python Software Foundation. The Python Tutorial. <https://docs.python.org/3/tutorial/>
- Python Software Foundation. Built-in Types — Lists and Dicts. <https://docs.python.org/3/library/stdtypes.html>
- Python Software Foundation. csv — CSV File Reading and Writing. <https://docs.python.org/3/library/csv.html>
- Python Software Foundation. Built-in Functions (sorted, max, min, sum, len, round). <https://docs.python.org/3/library/functions.html>
- Python Software Foundation. Errors and Exceptions (try/except). <https://docs.python.org/3/tutorial/errors.html>

Recursos complementarios

- Real Python. Python Dictionaries. <https://realpython.com/python-dicts/>
- Real Python. Reading and Writing CSV Files in Python. <https://realpython.com/python-csv/>
- Real Python. How to Use sorted() and .sort() in Python. <https://realpython.com/python-sort/>
- Real Python. Python Exceptions: An Introduction. <https://realpython.com/python-exceptions/>

Repositorio y entregables

- Repositorio GitHub: [nicolas-deprado/TPI_Prog1](https://github.com/nicolas-deprado/TPI_Prog1)
- Video demostrativo: https://drive.google.com/file/d/10zLza3BnUGBcS5DRNSWR9HNPZJi8NYoQ/view?usp=drive_link